Debugging Overdue Tasks Function

https://gemini.google.com/app/9ec382ff4007f8a3

User prompt: task_manager.py: ```python from datetime import date def add_task(tasks, title, due_date=None, priority="medium"): """Add a new task to the task list. Args: tasks (list): The list of existing task dictionaries. title (str): The task's title. due_date (date, optional): The task's due date. Defaults to None. priority (str): One of "high", "medium", "low". Defaults to "medium". Returns: list: The updated list of tasks. """ tasks.append({"title": title, "due_date": due_date, "priority": priority, "completed": False}) return tasks def mark_complete(tasks, title): """Mark a task as completed by title. Args: tasks (list): The list of task dictionaries. title (str): The title of the task to mark complete. Returns: bool: True if a matching task was found and updated, False otherwise. """ for task in tasks: if task["title"] == title: task["completed"] = True return True return False def get_overdue_tasks(tasks, current_date): overdue = [] for task in tasks: if task["due_date"] < current_date and not task["completed"]: overdue.append(task) return overdue def calculate_completion_rate(tasks): if not tasks: return 0.0 completed = sum(1 for t in tasks if t["completed"]) return completed / len(tasks) ``` Debug get_overdue_tasks for tasks whose due_date is None. First step back: briefly state the general principles for comparing optional dates and using short-circuit conditions. Then apply those principles to this function and provide the corrected function. Preserve incomplete tasks with dates strictly before current_date, exclude undated tasks, and do not modify the input. Keep the explanation concise.

Response: General Principles for Optional Dates and Short-Circuit Evaluation

1. **Avoid Incompatible Comparisons:** In Python, comparing `None` with a `datetime.date` object using relational operators (e.g., `<` or `>`) raises a `TypeError`. Any optional field must be verified as non-`None` before performing a comparison.
2. **Use Short-Circuit Evaluation:** Boolean expressions evaluate left-to-right and stop as soon as the outcome is determined. Placing `task["due_date"] is not None` before the relational check `task["due_date"] < current_date` ensures that Python only evaluates the comparison when `due_date` contains a valid date object.

Application to `get_overdue_tasks`

In `get_overdue_tasks`, tasks created without a `due_date` default to `None`. Evaluating `task["due_date"] < current_date` raises a `TypeError` on those undated tasks.

To fix this while keeping incomplete tasks with dates strictly before `current_date` and excluding undated tasks, update the condition to check that `due_date` is not `None` first:

```
def get_overdue_tasks(tasks, current_date):
    """Return a list of incomplete tasks whose due date is strictly before current_date.

    Tasks without a due date (due_date is None) are excluded.
    """
    overdue = []
    for task in tasks:
        if task["due_date"] is not None and task["due_date"] < current_date and not task["completed"]:
            overdue.append(task)
    return overdue
```
